Phase 1 Report

Data Collection & Tokenizer Construction

CL3410: Language Models and Agents, Monsoon 2026

Sourav Nath
Roll Number: 2025201054

19 August 2026

Models: Model H (Hindi, Devanagari) · Model L (Assamese, Bengali–Assamese script)

Status Note

This report covers dataset collection, cleaning, deduplication, splits, and tokenizer construction — all final and verified. All token counts below are **exact**, computed by tokenizing every train/val/test document with the actual selected tokenizer for its language (16k BPE for both), not estimated from a proxy fertility constant. Three corrections are folded into the numbers below. First, an early draft of this project used a calibration fertility from a throwaway tokenizer (1.463 Hindi, 1.688 Assamese) to steer collection targets in real time; once real tokenizers existed, exact counts came in measurably different, requiring a further expansion round to clear 500M on the real numbers (§6). Second, the vocabulary size initially selected by minimizing fertility alone (32k Hindi, 24k Assamese) was corrected down after course guidance clarified that the embedding layer counts against Phase 2's ∼25M-parameter budget — at 32k, Hindi's embedding matrix alone would have consumed up to 98% of that budget; the immediate fix moved both languages to 8k, the safest point in the course's explicitly recommended 5k–10k range. Third, once the same guidance was clarified as a recommendation rather than a strict rule, the selection was revisited once more on pure engineering merit and settled on **16k for both languages, with tied input/output embeddings** — keeping the embedding budget reasonable (24.6% of 25M tied) while recovering meaningfully better fertility and less subword fragmentation than 8k. §8.1 covers the full three-stage correction; every token count in this report reflects the final 16k tokenizers.

Corpus data (raw + cleaned, both languages):
https://drive.google.com/drive/folders/1jLpgQBmY3gGCWfqxP_Kz_4MQvQHxoGJS?usp=sharing

Contents

1 Language selection **2**

2 Data sources **2**
2.1 Manual sources 3
2.2 Downloaded sources 3

3 Cleaning pipeline **5**

3.1 A note on stage order 5

4 Deduplication 6

5 Splits 6

6 Manual vs. downloaded token split, and the central Assamese finding 7

7 Language identification validation (§4.2) 9

8 Tokenizer construction 10
8.1 Vocabulary size selection 10
8.2 Required statistics, on the selected vocabulary, measured on held-out text 11
8.3 Worked tokenization examples 12
8.4 Token-frequency distribution (Zipf pattern) 12

9 Key engineering findings 13

10 Repository layout 15

1 Language selection

Model H — Hindi. Chosen for its large public-corpus presence (Sangraha, IndicCorpV2) and the resulting ability to reach the ∼500M-token target from public sources within the project timeline.

Model L — Assamese. Chosen deliberately to maximize the *contrast* this project is meant to surface, not for convenience:

- **Script-sharing pressure.** Assamese shares the Bengali–Assamese Unicode block with Bengali, a much larger language, creating a genuine language-identification challenge — with a clean, cheap discriminator available (the Assamese/Bengali ৰ/র and ৱ/ব codepoint distinction), so the filter's precision can be demonstrated rather than assumed. Validated directly: **100% precision and recall on a 100-document hand-labelled set** (50 genuinely-Assamese documents from this project's own corpus, 50 genuinely-Bengali documents drawn independently from IndicCorpV2's Bengali portion — ground truth by source provenance, not by the classifier's own signals, to avoid a circular check).
- **Agglutinative morphology** → measurably higher tokenizer fertility than Hindi (1.6019 vs. 1.3067 tokens/word on each language's selected, final 16k tokenizer — §8.2), a direct, concrete finding for the fertility-comparison question Phase 3 asks.
- **Genuine data scarcity**, making the manual-vs-downloaded tension explored in §6 a real constraint rather than a manufactured exercise.

2 Data sources

Every document collected carries a `collection_method` field (`manual` or `downloaded`), tracked from the moment it enters the pipeline, per the course ruling: *"anything that involves you getting*

data, processing, cleaning and organizing is considered manual; anything already organized on HuggingFace which is then used is not manual."

2.1 Manual sources

Table 1: Manual data sources, by language

Language	Source	Method
Hindi	Wikipedia + Wikisource	Official XML dumps, own wikitext extractor
Hindi	News (`livehindustan`, `prabhatkhabar`, `jagran`)	Own sitemap-based scraper
Hindi	Archive.org books	Own OCR-text collector, `language:Hindi` facet
Assamese	Wikipedia + Wikisource	Official XML dumps, own wikitext extractor
Assamese	News (`asomiyapratidin`)	Own sitemap-based scraper — the only Assamese news source found to have a workable sitemap and meaningful volume after probing 20+ candidate domains across the project
Assamese	Archive.org books	Own OCR-text collector — both the `language:Assamese` (English-word) and `language:asm` (ISO-code) facets, after discovering the two are almost entirely disjoint and the original collector had only ever searched the first
Assamese	News (`xahitya.org`)	Own sitemap-based scraper, reusing the same scraper class as `asomiyapratidin` — collected independently of the pre-packaged `xahitya-assamese-corpus` dataset (§2.2) on the same site, which is what makes it manual rather than downloaded under the course ruling

Wikipedia/Wikisource are collected via the **official XML dumps with a self-written wikitext extractor**, not by scraping the live site — scraping was permitted by the course ruling ("yes you can scrape"), but dump parsing is the same act of collection done more completely and without hammering Wikimedia's servers.

2.2 Downloaded sources

Table 2: Downloaded data sources, by language

Language	Source	Notes
Hindi	Sangraha (AI4Bharat), `verified` config	Redrawn once at a higher cap (400M raw words, was 345M) after the fertility correction showed the first draw fell short on exact tokenizer counts; `unverified` never needed

Table 2: Downloaded data sources, by language

Language	Source	Notes
Assamese	Sangraha (AI4Bharat), `verified` + `unverified`	`verified` naturally exhausted (157.46M raw words — the entire non-synthetic supply for Assamese); `unverified` is Sangraha's own smallest, lowest-priority tier
Assamese	IndicCorpV2 (AI4Bharat)	Flat corpus file, fully drawn (∼67.3M raw words)
Assamese	MADLAD-400 (AllenAI), `clean` + `noisy` splits	Both fully drawn; `noisy` measured to have lost ∼79% of its raw content to deduplication already, genuinely lower marginal value than `clean` but still net-positive once manual growth (§6) removed the need to trim it
Assamese	Glot500 (cis-lmu), all 9 sub-tags	An aggregation corpus; used specifically because it was the only practical route to two sources tried directly and blocked (OSCAR — gated behind HuggingFace authentication; CC100 — no longer hosted at its documented path). The `Indiccorp` sub-tag measured 99.9%+ dedup survival despite the name overlap with the IndicCorpV2 source above — confirmed genuinely distinct content, not a repackaging
Assamese	xahitya-assamese-corpus (Ranjit89, HF)	A scrape of xahitya.org (Assamese literature); verified genuine before use, but measured to overlap existing sources far more than its specialist framing suggested (32.5% dedup survival) — much of its unique content was absorbed into the manual xahitya.org scrape (§2.1) via deduplication's survivor-selection rule instead
Assamese	assamese-monolingual-corpus (MWirelabs, HF)	Pre-segmented sentence CSV; verified genuine, and measured to be the most valuable single downloaded addition of the later expansion rounds (97.2% dedup survival — largely non-overlapping with everything already collected)

Every downloaded source drawn and measured across the project is present in the final corpus. Earlier rounds temporarily trimmed several of these out to protect the 20% manual floor while manual supply was smaller (§6); the final round's manual growth raised the achievable ceiling enough that all of them could be safely re-included, so no source measured genuine and kept clean needed to be discarded in the end.

Sangraha's `synthetic` config (machine-translated/generated text) was never used for either language, per the explicit course ruling against it. A live check of Sangraha's own data found mislabelled rows in both languages (documents that were not actually in the target language) — every downloaded row is passed through the identical language-identification stage as manually-collected

text, not assumed clean by provenance.

3 Cleaning pipeline

Six stages, run identically across every source (only per-source thresholds differ, see §3.1):

1. **Normalize** — Unicode NFC, selective ZWJ/ZWNJ handling, digit convention, mojibake repair (`ftfy`), control-character stripping.
2. **Language identification** — a script-ratio gate followed by a function-word/codepoint discriminator per language pair (Hindi vs. Marathi/Nepali/Bhojpuri/Sanskrit; Assamese vs. Bengali). Book-like sources (Wikisource, archive.org) get paragraph-level repair instead of whole-document rejection, since a single non-target-language front page or appendix would otherwise cost an entire book.
3. **Boilerplate/quality heuristics** — dateline and UI-furniture stripping, length bounds, symbol-to-letter ratio, line-structure filtering (catches word-list "books" that are dictionaries, not prose), PII anonymisation.
4. **Non-language word removal** — a word→sentence→document cascade that strips embedded Latin-script text (per the ruling *"digits are fine but do remove non-language words"*) while keeping digits and in-script text untouched.
5. **Machine-translation exclusion** (Wikipedia only) — dump-level detection of translation-flag templates and edit-summary markers, a TA-mandated filter; measured prevalence (a lower bound, since dumps carry only the latest revision) was 1.3% for Hindi Wikipedia, 0.2% for Assamese.
6. **Deduplication** — see §4.

Thresholds are deliberately asymmetric between languages (the judgement the course explicitly assesses, per Lecture 3 slide 30): Hindi, with a data surplus, filters hard and favours precision; Assamese, with genuine scarcity, filters gently and favours recall, repairing documents (stripping the offending portion) rather than discarding them where reasonable. Concretely: Hindi news lost 14.4% of its words to the boilerplate/quality stage (mostly datelines) versus 0.96% for Assamese news, under thresholds tuned by reading actual rejected documents in each language rather than importing one set of numbers.

3.1 A note on stage order

The non-language-word cascade must run **after** boilerplate removal, not before — running it first measured a 64% document-loss rate on Hindi news, because datelines and bylines (`IST`, `Aug`, `pm`) were being counted as "non-language words" rather than the code-mixing the stage is meant to catch. Reordering the stages (boilerplate → non-language words, not the reverse) fixed this without loosening any threshold. This is recorded because it is exactly the kind of pipeline-design decision the course asks to be justified, not just implemented.

4 Deduplication

Three passes, matching the taught method: **exact** (SHA-256 of normalised text) → **paragraph** (cross-document boilerplate, a paragraph repeated in ≥5 documents is dropped everywhere) → **near-duplicate** (MinHash + LSH, $k = 100$ hashes, $b = 20$ bands, threshold ≈ 0.55, 3-word shingles capped at 3000 per document to keep cost independent of document length).

Final dedup pass (all sources merged, run once per language on the true final corpus, after the fertility-proxy correction's final expansion round):

Table 3: Final dedup pass, all sources merged, per language

	Docs in	Words in	Docs out	Words out	Retention
Hindi	1,039,530	476,891,399	1,010,907	464,048,535	97.3%
Assamese	929,935	436,924,957	846,348	384,237,720	87.9%

Assamese's lower retention reflects genuine overlap between its many downloaded sources (Sangraha, IndicCorpV2, MADLAD, Glot500, and two further sources added across the project's expansion rounds all draw, to varying degrees, from overlapping underlying web/corpus lineages) — the dedup pass is precisely what makes that overlap harmless rather than double-counted. Assamese's corpus went through **six expansion rounds** in total across the project as new sources were found and measured, and a further manual-collection push in the final round, each verified genuine before use rather than assumed. This final round's dedup pass reused 899,592 of 900,017 document signatures from a persisted cache (99.95%), finishing in about 37 minutes rather than the 2–3 hours a cold run took earlier in the project.

Decontamination (test-vs-train near-duplicate check, distinct from the corpus-internal pass above): indexes the train split, queries it with every validation and test document's MinHash, and drops any train document that near-duplicates a held-out one. Run separately from ordinary dedup because ordinary dedup can still leave a near-duplicate pair split across train and test — it collapses a cluster to one survivor before any split label exists, so the surviving copy can land on either side of the boundary by chance.

Both languages: complete, zero leakage found on every corpus checked across the whole phase, including the true final corpus. Final runs, on the corpus this report's numbers are drawn from: 0 of 989,016 Hindi train documents and 0 of 824,317 Assamese train documents near-duplicated anything in validation or test. Hindi's decontamination indexed in 40.2 minutes; Assamese's in 22.4 minutes.

5 Splits

Deterministic, hash-based assignment (SHA-256 of document id → bucket), computed once and frozen — not a stateful process, so it is insensitive to collection order and reproducible from the corpus alone.

Both languages' validation and test sets target 5M words each, computed directly from that word count rather than a fixed percentage — landing close to it for both (Hindi 5.34M/5.23M, Assamese 5.21M/5.12M), comfortably above the ∼2M-word floor needed for a stable perplexity estimate.

Assamese's split sizing changed twice across the project as the corpus grew; Hindi's

Table 4: Train/validation/test splits, per language (train tokens are exact counts on the selected 16k tokenizer)

	Train docs	Train words	Train tokens	Val words	Test words
Hindi	989,016	453,479,963	**589,952,668**	5,342,102	5,226,470
Assamese	824,317	373,902,951	**591,255,819**	5,212,836	5,121,933

was aligned to match at the end. Assamese was originally 5%/5%, sized when the corpus was estimated at only ∼60–90M tokens; once the corpus passed 500M tokens that fraction was needlessly withholding tens of millions of words from train, and was reduced to 1%/1%, then generalised to target an absolute 5M words per split directly (for evaluation-set-size consistency regardless of how either corpus's total changed). Hindi was already close to 5M/5M under its original 1%/1% fraction (4.31M/4.53M) and was moved to the same exact-word-count scheme for consistency between the two languages.

6 Manual vs. downloaded token split, and the central Assamese finding

Per the course requirement (≥20% of final training tokens must be manually collected, for both languages), measured **exactly** — every train document tokenized with its language's real selected tokenizer, tokens attributed by each document's `collection_method`, not estimated from a word count and a fertility constant:

Table 5: Manual vs. downloaded train tokens, per language

	Manual tokens	Downloaded tokens	Total tokens	Manual %
Hindi	122,696,450	467,256,218	589,952,668	**20.80%**
Assamese	141,178,394	450,077,425	591,255,819	**23.88%**

Both languages clear both targets with real margin. Getting here took six expansion rounds for Assamese and a late, project-wide correction that affected Hindi too — and that corrective process is itself the most important finding of this phase, not an incidental detail.

The fertility-proxy error, and why it mattered more for Assamese than Hindi

For most of the project, token counts were tracked using a calibration fertility measured once, early, from a throwaway SentencePiece model (1.463 Hindi, 1.688 Assamese tokens/word). Five successive Assamese expansion rounds were planned and evaluated against that proxy, each one landing the corpus just over 500M **proxy** tokens with the 20% floor protected. Only once the real tokenizers existed (§8) and were used to tokenize the corpus exactly did it become clear the proxy had been overstating real fertility throughout: the real, measured fertility on the selected tokenizers is **1.2402 Hindi, 1.4913 Assamese** (§8.2) — both lower, meaning every "final" token count reported up to that point was an overestimate. Exact recount on the then-current corpora: Hindi 494,966,479 train tokens (short of 500M by ∼5.0M), Assamese 451,001,443 (short by ∼49.0M). Neither language's manual ratio was ever at risk — this was purely a token-count miscalibration, not a compliance problem — but both now needed a further expansion round to clear 500M on the numbers that

actually count.

Assamese's gap was structurally the harder of the two to close. With manual fixed at its then-current 97.88M tokens, the achievable ceiling on total corpus size at the 20% floor (`manual / 0.20`) was itself only ∼489.4M tokens — below 500M even before subtracting anything for validation/test, a tighter version of the same manual-scarcity constraint documented in earlier rounds of this project: the 20% floor caps total corpus size at `manual / 0.20`, a ceiling only manual growth can move, and Assamese's manual supply had appeared close to exhausted after five rounds of searching. Hindi's fix, by contrast, was mechanical — real ratio headroom existed, so redrawing the downloaded tier at a higher cap (400M raw words, was 345M) was sufficient.

Closing Assamese's gap: real manual growth, and a genuine limit of archive.org's own API

The only lever available was more manual collection. Archive.org's book pool under the `language:asm` (ISO-code) facet — already known from an earlier round (§9) to be far larger and almost entirely disjoint from the `language:Assamese` facet — had previously been resumed but was itself far from exhausted (a prior session had processed only ∼3,400 of the facet's advertised items before stopping on a self-imposed item cap, not a real limit). It was resumed twice more this round, deliberately targeting the facet's full advertised size:

- **First pass**: hit an inherited item cap cleanly (not a crash) after adding 962 new books, ∼31.6M raw words.
- **Second pass**: deliberately set to cover the facet's entire advertised pool — archive.org's own search API reports **15,385 items** matching this query via its `numFound` field, verified by a direct API check. The pass nonetheless stopped after scanning only **∼3,300 items**, well short of that count. This is a genuine, previously-undocumented limitation of archive.org's `advancedsearch.php` endpoint: its deep pagination (`page=N`) silently returns an empty result page past a few thousand results rather than erroring or reporting a real end, regardless of what `numFound` claims is available. **Worth stating plainly as a finding: `numFound` on this API is not a reliable bound on what is actually retrievable through paginated search** — a caveat any future use of this collector should carry.

Combined yield across both passes: **1,451 books total, 55.87M raw words** (up from 62 books / 1.8M words before this round) — the single largest manual addition of the project. Cleaned and merged via a full dedup rerun across all eleven Assamese source directories — including the four downloaded sources trimmed out in an earlier round to protect the (then-lower) manual ceiling (MADLAD's `clean` and `noisy` splits, Glot500's eight non-`Indiccorp` sub-tags, and Sangraha's `unverified` tier) — this landed a result of 559,672,837 exact train tokens at 23.97% manual (measured on the 32k/24k tokenizers then in use), the best margin on both the token target and the manual floor achieved up to that point, and the first round where re-including every previously-trimmed downloaded source was safe rather than ratio-breaking, precisely because manual growth had raised the ceiling past what that volume needed.

This is a more complete version of the outcome the course materials anticipate for a genuinely low-resource language (*"you will not reach [500M] for your low-resource language… what you DO and LEARN in that context is the point of the exercise"*): the project's working assumption for most of its timeline, based on proxy numbers, was that Assamese would land just barely over 500M

or need an honest, documented shortfall. The real, exact numbers turned out better than that once the fertility error was caught and corrected — but catching that error, and the six rounds of empirical, measure-don't-assume work needed to close the resulting gap without touching the 20% floor, is the more precise version of the intended finding for this corpus than either the original optimistic estimate or the pessimistic one that briefly replaced it.

A second and third correction, layered on top: getting the vocabulary size right took two more passes

The numbers earlier in this section used the 32k (Hindi) / 24k (Assamese) tokenizers selected in §8.1's original fertility-minimization sweep. Course guidance subsequently clarified a constraint that sweep hadn't weighted heavily enough: the embedding layer counts against Phase 2's ∼25M-parameter budget, and at 32k vocabulary with $d_{\text{model}} = 384$, Hindi's embedding matrix alone consumes **49% of that budget tied, 98% untied** — leaving too little capacity for the Transformer layers actually responsible for language modelling and reasoning. The immediate fix moved both languages to **8k vocabulary**, the safest point inside the course's stated 5k–10k range, and retokenizing the identical final corpus at 8k gave 643,241,614 Hindi tokens (20.85% manual) and 663,584,501 Assamese tokens (23.64% manual) — comfortably clear of target, but with meaningfully higher fertility and more subword fragmentation than the original sweep's larger sizes (§8.3).

Once it was clarified that this guidance was a recommendation rather than a strict rule, the choice was revisited a third time on engineering merit alone rather than compliance safety. §8.1 covers the reasoning in full; the outcome was **16k vocabulary for both languages, paired with tied input/output embeddings** — a deliberate middle point that keeps the embedding budget reasonable (24.6% of 25M tied, versus 8k's 12.3%) while meaningfully recovering fertility and reducing fragmentation relative to 8k, at a cost that stays well clear of the 32k/24k-scale problem that started this whole correction chain.

Because fertility still rises as vocabulary shrinks (just less steeply than at 8k), retokenizing the identical final corpus with the 16k tokenizers gives the numbers in the table above: **589,952,668 Hindi tokens (20.80% manual)** and **591,255,819 Assamese tokens (23.88% manual)** — both comfortably clear of 500M and the 20% floor, with Assamese's manual share still the higher of the two. This is the true final state of the corpus; the 32k/24k- and 8k-based figures earlier in this section describe genuine, accurately-measured history, not numbers that were ever wrong for the tokenizer in use at each point.

7 Language identification validation (§4.2)

The Assamese/Bengali discriminator (script-ratio gate + ৰ/র and ৱ/ব codepoint counts + function-word rates) was validated against a 100-document set with ground truth established by **source provenance**, not by the classifier's own signal (avoiding a circular check): 50 documents drawn from this project's own Assamese corpus, 50 drawn independently from IndicCorpV2's Bengali portion.

Result: precision 1.000, recall 1.000, F1 1.000 (50/50 true positives, 0 false positives, 0 false negatives). Spot-checked evidence confirms the classifier is using genuine signal, not an artifact: true-Assamese documents show a strong ৰ-vs-র skew in one direction (e.g. 171 vs. 16 in one sample),

true-Bengali documents show the opposite (0 vs. 157 in another) — the discriminator's two markers behave exactly as designed on both sides of the boundary.

8 Tokenizer construction

Two fully independent SentencePiece BPE tokenizers, trained from scratch on each language's train split only (fitting on held-out text would leak split boundaries into the vocabulary itself). `character_coverage=0.9995`, `byte_fallback=True`, `input_sentence_size=4,000,000` with shuffling (SentencePiece's own answer to training on a 500M+-token corpus without holding it all in memory — it subsamples). Training text is sentence-segmented on the Devanagari/Bengali danda (।) plus residual Latin punctuation, not fed one document per line, because `max_sentence_length` (4192 bytes by default) would otherwise silently truncate long documents.

8.1 Vocabulary size selection

Swept $\{8k, 16k, 24k, 32k, 48k, 64k\}$ per language, measuring fertility (tokens/word) and byte-fallback rate on held-out (validation-split) text at each size:

Table 6: Hindi vocabulary sweep — **16,000** (highlighted) is the corrected selection

Hindi vocab	Fertility	Bytes/tok	Byte-fallback rate
8,000	1.432	9.20	0.348%
16,000	**1.313**	**10.02**	**0.380%**
24,000	1.266	10.40	0.394%
32,000	1.240	10.62	0.402%
48,000	1.212	10.86	0.412%
64,000	1.197	11.00	0.417%

Table 7: Assamese vocabulary sweep — **16,000** (highlighted) is the corrected selection

Assamese vocab	Fertility	Bytes/tok	Byte-fallback rate
8,000	1.769	9.60	0.545%
16,000	**1.575**	**10.78**	**0.612%**
24,000	1.491	11.39	0.647%
32,000	1.444	11.77	0.668%
48,000	1.390	12.22	0.694%
64,000	1.359	12.50	0.710%

Fertility decreases monotonically with vocabulary size for both languages, so minimizing fertility alone always selects the largest size swept (64k) — this is not the right selection criterion in isolation. Phase 2 fixes each model at ∼25M trainable parameters, and the embedding matrix costs vocab_size × d_{model}: at a plausible $d_{\text{model}} = 384$, a 64k vocabulary would consume **98.3% of the entire parameter budget** before a single Transformer layer exists, against a fertility gain over 32k of only ∼3.5% relative (1.240 → 1.197). This trade-off was computed explicitly, not assumed, before selecting.

First selection, and why it was revised. The reasoning above initially led to picking each lan-

guage's point of *diminishing* fertility returns — Hindi 32k, Assamese 24k — treating the embedding-cost check as a sanity bound against the most extreme sizes (64k) rather than the binding constraint on the sizes actually selected. That was the wrong frame: at $d_{\text{model}} = 384$, Hindi's selected 32k vocabulary alone already consumes **49% of the 25M-parameter budget with tied input/output embeddings, and 98% if untied** — functionally the same problem as the rejected 64k case, just one step removed. Course guidance made this explicit directly: the embedding layer counts fully against the parameter budget, and a vocabulary in the tens of thousands leaves too little capacity for the Transformer layers actually responsible for language modelling and reasoning quality; the recommended range is **5k–10k**, with sizes a little over 10k acceptable but 24k/32k specifically flagged as too large.

Second selection: 8k, the conservative fix. As the safest point inside the stated 5k–10k range, both languages were moved to 8k — already the smallest size in the original sweep, so this needed no retraining beyond its no-fallback variant. At $d_{\text{model}} = 384$, 8k costs **12.3% of the 25M budget tied (24.6% untied)**. The real cost: fertility rose measurably above 32k/24k's (1.4247 vs. 1.2402 Hindi, 1.7969 vs. 1.4913 Assamese) and subword fragmentation increased visibly (§8.3).

Third selection (final): 16k for both languages, with tied embeddings. The 5k–10k guidance was subsequently clarified as a recommendation, not a hard rule, which reopened the question on pure engineering grounds. Two considerations pushed back toward a somewhat larger vocabulary than 8k: first, at a fixed context length, higher fertility means the model sees *less real text* per forward pass — 8k's fertility costs roughly 9% of effective context compared to 16k's, which matters directly for the multi-entity comparative-reasoning tasks Phase 3 will fine-tune on. Second, a 25M-parameter model has limited depth to spare for stitching sub-word fragments back into whole-word meaning, so reducing unnecessary fragmentation lightens a real burden on the model's actual capacity, not just the embedding table. Weighed against this: at $d_{\text{model}} = 384$, 16k costs **24.6% of the 25M budget tied (49.2% untied)** — safely inside a reasonable range provided the design commits to tied embeddings, which this project does. That decision was made explicitly for that reason: it keeps 16k's embedding cost at the same order of magnitude as 8k's *untied* cost, meaning the parameter-budget concern that started this whole correction is still respected, just with a different, deliberate trade-off between embedding size and subword granularity.

Selected: 16k for both languages. Already present in the original sweep (no new byte-fallback training needed, only the no-fallback variant for the true UNK-rate stat, §8.2). Fertility at 16k (1.3067 Hindi, 1.6019 Assamese) sits meaningfully below 8k's and only modestly above 32k/24k's — most of the fragmentation cost of 8k is recovered without paying anything close to 32k/24k's embedding-budget cost. Both languages again land on the *same* selected size, since the parameter-budget ceiling is a language-independent constraint; the fertility *comparison* itself (Assamese consistently higher at every swept size) remains the substantive cross-language finding.

8.2 Required statistics, on the selected vocabulary, measured on held-out text

On byte-fallback and UNK, measured properly rather than reported as a degenerate zero. With `byte_fallback=True`, the SentencePiece-reported UNK rate is trivially ∼0% by construction — bytes that don't map to a known piece get spelled out as raw-byte tokens instead of `<unk>`, which makes "UNK rate" meaningless as reported. Both a byte-fallback and a no-fallback variant of each selected tokenizer were trained; the no-fallback variant's real out-of-vocabulary rate (0.119% Hindi, 0.185% Assamese) is the honest number, and the byte-fallback trigger rate above

Table 8: Required tokenizer statistics, selected 16k vocabulary, measured on held-out text

	Hindi (16k)	Assamese (16k)
Fertility (tokens/word)	1.3067	1.6019
Characters per token	3.929	4.019
Byte-fallback trigger rate	0.429%	0.650%
True UNK rate (no-fallback variant)	0.119%	0.185%
Distinct tokens used (of vocab)	15,861 / 16,000 (99.1%)	15,880 / 16,000 (99.25%)
Singleton tokens (used exactly once)	61 (0.38% of used)	10 (0.06% of used)

is its proxy in the deployed (fallback-enabled) model. Both rates sit between 8k's and 32k/24k's, as expected — coverage tracks vocabulary size smoothly, with `character_coverage=0.9995` and byte-fallback unchanged throughout. Assamese's higher rate on both metrics is consistent with its higher fertility and smaller training corpus.

Vocabulary utilization is near-complete for both (99.1% and 99.25% of the trained vocabulary actually appears in held-out text), meaning 16k was not oversized for either corpus — almost every trained piece earns its place.

8.3 Worked tokenization examples

Table 9: Worked tokenization example, selected 16k tokenizer

Hindi	Assamese
नमस्ते, मेरा नाम राजेश है और मैं भारत से हूँ।	নমস্কাৰ, মোৰ নাম ৰাজেশ আৰু মই অসমৰ পৰা আহিছোঁ।
→ _नम स्ते , _मेरा _नाम _राजेश _है _और _मैं _भारत _से _हूँ । (13 tokens)	→ _নমস্কাৰ , _মোৰ _নাম _ৰাজেশ _আৰু _মই _অসমৰ _পৰা _আহিছোঁ । (11 tokens)

Note: "_" above stands in for SentencePiece's word-boundary marker (rendered as U+2581 "_" in the original tool output), marking a preceding space.

16k recovers most of the whole-word fragmentation seen at 8k, but asymmetrically between languages: Assamese's example is fully back to whole-word tokens (" ", " ", " " each single pieces again, matching the original 24k-era tokenization exactly), while Hindi's " " (namaste) still splits as _ + — this specific word's BPE merge apparently needs more than 16k vocabulary budget to earn a dedicated token, a concrete illustration that "vocabulary size" and "fertility" don't translate into identical per-word coverage across languages even at the same nominal size. High-frequency function words (_ , _ , _ , _) tokenize as single pieces at every vocabulary size tested, since those are frequent enough to earn a dedicated token even in the smallest vocabulary — the fragmentation cost falls disproportionately on less-common content words and names, and how much of it remains at a given size is genuinely language-specific.

8.4 Token-frequency distribution (Zipf pattern)

Top tokens on held-out text follow the expected Zipfian shape — a small number of function words dominate:

Hindi top 10: ।(214,715) के(209,089, "of/for") है(158,908, "is") में(156,519, "in") ,(151,410) की(121,828, "of") से(92,013, "from") और(90,905, "and") को(88,386, obj. marker) का(79,209, "of")

Assamese top 10: ।(241,130) ,(172,739) আৰু(84,839, "and") -(79,478) '(49,502, apostrophe/quote) এই(45,105, "this") কৰি(37,641, "doing") '(37,320, right single quote variant) ।(36,946) ৰ(35,777, genitive/possessive suffix piece)

Both distributions are dominated by genuine high-frequency function words and punctuation, not artifacts — confirming the tokenizers learned real linguistic structure rather than degenerate byte-level splitting. Assamese's list still carries one single-character suffix piece (ৰ, genitive/possessive), a milder version of the fragmentation symptom seen more heavily at 8k — most of the list has reverted to genuine whole words and punctuation, consistent with §8.3's finding that 16k recovers most (though not all) of 8k's fragmentation.

9 Key engineering findings

A selection of the more consequential bugs found and fixed this phase, each caught by direct measurement rather than assumption:

- **Document-id collisions across collector restarts** corrupted an early split by assigning colliding documents inconsistently. Fixed by deriving every id from a stable content hash (URL, archive.org identifier, or upstream doc_id) instead of a per-run counter.
- **A resumable dedup output directory silently duplicated an entire prior run's output** when dedup was re-run after adding new sources, because the writer's own resumability (continue numbering after existing shards) was the wrong behaviour for a function that recomputes from scratch every call. Caught via a routine post-run id-uniqueness check, not a crash.
- **"Hindi clears 500M" was initially measured against the wrong number** — the full corpus (train+val+test) instead of the training split specifically, which the spec's wording ("500M *training* tokens") requires. The error was caught by a direct re-measurement prompted by a sanity-check question, not by re-reading the spec.
- **A structural gap in the archive.org book collector**: it only ever searched the `language:Assamese` facet (2,576 items) and never `language:asm`, the ISO-code variant — a nearly 6× larger, almost entirely disjoint pool (15,385 items) that had simply never been queried. Found by comparing facet counts directly rather than assuming the original search was exhaustive.
- **An out-of-memory crash** from running two full corpus-wide deduplication passes concurrently on an 8GB-RAM machine, one of which was loading a large in-memory signature cache. Recovered without data loss (all prior results were safely on disk); resolved by not running two heavy dedup passes concurrently going forward.
- **A silently stalled network stream** during a large (655MB) source download: the process stayed alive with no error, but genuine progress was near-zero (23 seconds of real CPU time over 5.5 hours of wall-clock). Caught by checking actual CPU utilization rather than trusting that "still running" meant "still working"; restarting from scratch resolved it and completed normally.

- **"More downloaded data will raise the token ceiling" was tested directly and found false once manual is fixed.** After one round pushed Assamese past 500M tokens but broke the 20% manual floor, a second, larger round added 47M more raw downloaded words specifically to get real margin — and manual fraction dropped *further* (16.96%, worse than the first breach at 17.79%), because the achievable ceiling is `manual / 0.20`, a quantity downloaded volume cannot move. This was projected numerically before running the round (every retention scenario from 30–90% predicted a sub-20% result) and confirmed exactly by the real outcome — the kind of check-before-you-run discipline that turned a potentially wasted multi-hour data-collection effort into a validated structural finding instead.
- **A correction applied to a derived output rather than to the pipeline stage that produces it does not survive that stage being rerun.** A corpus trim (dropping low-value sources to restore the manual floor) was applied directly to deduplication's final output; the next time new data required a full dedup rerun, the pass read from the untouched source directories underneath and silently regenerated the untrimmed corpus, undoing the correction. Caught immediately by comparing per-source totals against what had deliberately been removed, not by a crash.
- **The signature-cache question from earlier in the project (0 cache hits on one run, unexplained) was resolved by more data, not more debugging.** Several subsequent full-scale runs reused 87%, 95.8%, 99%, and (on runs where the underlying corpus was completely unchanged) 100% of cached signatures respectively — strong evidence the original 0-hit run was an isolated incident tied to that specific run's circumstances (it followed an out-of-memory crash and restart) rather than a defect in the caching design itself.
- **A calibration constant measured once, early, was trusted for an entire multi-day collection effort without being re-verified against the real artifact it stood in for — and it was wrong.** Every "final" token count reported before the tokenizer sweep completed used a proxy fertility from a throwaway model (§6). The real, exact fertility measured on the actual selected tokenizers came in lower for both languages, meaning every prior figure was an overestimate — caught only by tokenizing the corpus exactly once the real tokenizers existed, not by re-checking the proxy earlier. The plan's own stated discipline ("after the final tokenizer is selected, re-count everything for real") was correct; the lesson is that it should have been sanity-checked against a partial real tokenizer earlier rather than left until the very end of collection.
- **`numFound` on archive.org's search API is not a reliable bound on what is actually retrievable.** Closing Assamese's post-correction gap meant resuming book collection against a facet the API itself reports as having 15,385 matching items — verified directly, not assumed. The collector's pagination nonetheless exhausted after reaching only ∼3,300 items, a real limitation of the endpoint's deep-pagination behaviour (it returns an empty page well short of the claimed total rather than erroring), not a bug in this project's code. Worth documenting for any future reuse of this collector rather than re-discovering it.
- **A fertility-minimizing vocab-size selection under-weighted the actual binding constraint: Phase 2's fixed parameter budget.** §8.1's original sweep-based selection (Hindi 32k, Assamese 24k) picked each language's point of diminishing fertility returns, treating embedding cost as a check against only the most extreme swept size (64k). It was not: at $d_{\text{model}} = 384$,

the *selected* 32k vocabulary alone already consumed 49% of the 25M-parameter budget with tied embeddings (98% untied) — essentially the same problem as the 64k case it was chosen specifically to avoid. Course guidance made the constraint explicit (embeddings count fully against the budget; 5k–10k recommended), which meant revisiting the selection rather than just the framing. Because 8k had already been swept and measured, no retraining was needed for the byte-fallback model, only its no-fallback variant — but every downstream number (exact token counts, manual/downloaded split, worked examples, frequency lists) had to be recomputed against the new tokenizer. **General lesson tying this to the fertility-proxy finding above: a selection criterion computed correctly for the wrong candidates (the extremes swept, not the ones actually chosen) produces the same class of silent error as a correct calculation on a stale number — both require checking the specific claim being made, not a nearby one that looks similar.**

- **A recommendation followed as a strict rule, then correctly reopened once its actual status was clarified, without discarding the reasoning that motivated it.** The 8k selection above was the safest choice under the assumption that "5k-10k" was a hard requirement. Once clarified as a recommendation, the honest move was not to revert to the original (already-shown-flawed) 32k/24k selection, but to re-run the same cost-benefit analysis on genuine engineering merit: 16k costs 24.6% of the parameter budget tied — the same order of magnitude as 8k's *untied* cost, so the original concern is still respected — while recovering most of 8k's fertility/fragmentation penalty (§8.3 shows Assamese's worked example returning to full whole-word tokenization at 16k, though Hindi's does not, a genuinely language-specific asymmetry). The lesson: a course recommendation's *reasoning* remains binding even after its *strictness* is relaxed — the fix was to satisfy the reasoning better, not to abandon it.

Full details, including the exact measurements behind each finding, are kept in the project's working plan document for reference during the viva.

10 Repository layout

```
common/            # shared pipeline code (never data) -- normalize, filters, dedup,
                   # decontaminate, pipeline orchestration, tokenizer training
hindi/              assamese/
splits/             splits/       # val.ids, test.ids committed; train.ids
stats/              stats/        #   regenerable (corpus minus val/test), on Drive
tokenizer/          tokenizer/    # *.model, *.vocab committed (small, ~1MB)
report/
phase1_report.pdf         (this file)
```

`train.ids` for both languages is deliberately not committed to git — at final corpus scale it reached 41.7MB (Hindi) / 20.5MB (Assamese), large enough to make `git push` time out, and it is fully regenerable from the corpus plus the (small, committed) `val.ids`/`test.ids`, since split assignment is a pure deterministic hash rather than a stateful process. It is included on the Google Drive upload alongside the corpus text instead.

Data availability

Raw and cleaned corpus text (both languages) plus `train.ids` are hosted on Google Drive, permissioned for access without a request: https://drive.google.com/drive/folders/1jLpgQBmY3gGCWfqxP_Kz_4MQvQHxoGJS?usp=sharing. Tokenizer model/vocabulary files, split id manifests (`val.ids`/`test.ids`), and all statistics are committed directly in this repository (see §8 and §5) since they are small enough not to require Drive.